Concurrencia

Transacción: secuencia ordenada de instrucciones atómicas

Concurrencia: ejecutar múltiples transacciones en forma simultánea.

Concurrencia solapada: ejecución de múltiples transacciones en el mismo periodo de tiempo, donde sus instrucciones atómicas se intercalan sin ejecutarse exactamente al mismo tiempo.

Instrucciones básicas: se va a operar sobre ítems: el tamaño de ítem escogido se conoce como granularidad.

- read(X): lee el valor del ítem X, cargándolo en una variable en memoria
- write(X): ordena escribir el valor que está en memoria del ítem X en la base de datos

Propiedades ACID

La ejecución de transacciones debería cumplir con 4 propiedades deseables:

- **Atomicidad**: Desde el punto de vista del usuario, las transacciones deben ejecutarse de manera atómica. Esto quiere decir que, o bien la transacción se realiza por completo, o bien no se realiza.
- **Consistencia**: Se debe preservar la consistencia de los datos. La consistencia se define a través de reglas de integridad: condiciones que deben verificarse sobre los datos en todo momento
- **Aislamiento**: El resultado de la ejecución concurrente debe ser el mismo que si se ejecutase de forma serial. La ejecución concurrente debe ser equivalente a una serial.
- **Durabilidad**: Cuando se complete una transacción, se debe garantizar la persistencia de la misma, independientemente de cualquier falla que pueda ocurrir.

Para garantizar las propiedades ACID, los SGBD disponen de mecanismos de recuperación que permiten deshacer/rehacer una transacción en caso de que se produzca un error o falla.
Para ello se usan las instrucciones:

- begin: indica el comienzo de la transacción.
- commit: indica que la transacción terminó exitosamente, y se espera que el resultado se haya persistido.
- abort: indica que se produjo un error o falla y que entonces todos los efectos de la transacción deben ser deshechos.

Anomalías

Situaciones que violan las propiedades ACID.

Lectura sucia

Ocurre cuando una transacción lee un dato que fue modificado por otra transacción que todavía no fue confirmada. Si esa transacción luego se deshace, la lectura no tiene validez.
Es un conflicto de tipo WR: W_{T1} (X)...R_{T2} (X)...(a_{T1} ó c_{T1})

Actualización perdida

Una transacción sobrescribe un dato que otra había leído y aún no había terminado. El cambio de la segunda se pierde si la primera luego también escribe.
Es un conflicto de tipo RW, seguido por otro de tipo WW.

Lectura no repetible

Una transacción lee un valor, otra lo modifica, y al volver a leerlo, el valor ya cambió.
Es un conflicto de tipo RW, seguido por otro de tipo WR.

Anomalía del fantasma (phantom)

Ocurre cuando una transacción T_1 lee un conjunto de datos que cumplen una condición, pero otra transacción T_2 modifica ese conjunto (agregando, borrando o cambiando ítems) mientras la primera aún está en curso. Así, el conjunto leído puede cambiar inesperadamente.
Atenta contra la serializabilidad y suele requerir locks para resolverla.

Serializabilidad

Sea la siguiente notación:

- R_T(X): la transacción T lee el ítem X
- W_T(X): la transacción T escribe el ítem X
- b_T: begin de la transacción T
- c_T: commit de la transacción T
- a_T: abort de la transacción T
- m(T): cantidad de instrucciones de T

Solapamiento

Un solapamiento es una secuencia entrelazada de instrucciones de dos transacciones donde se respeta el orden interno de cada transacción.
Se analiza si cada solapamiento es **serializable**, es decir, si equivale a una ejecución en serie.

Ejecución serial

Dado un conjunto de transacciones T1, T2, ..., Tn una ejecución serial es aquella en que las transacciones se ejecutan por completo una detrás de otra, en base a algún orden Ti1 , Ti2 , ..., Tin .

Serializabilidad

Un solapamiento es serializable si cuando la ejecución de sus instrucciones en dicho orden deja a la base de datos en un estado equivalente a aquel en que la hubiera dejado una ejecución serial.
Que un solapamiento sea serializable garantiza la propiedad de aislamiento.

Equivalencia entre órdenes de ejecución: hay de resultados, de vistas y

- Equivalencia de conflictos: cuando ambos órdenes de ejecución tienen los mismos conflictos entre instrucciones: es la más fuerte.

Equivalencia de conflictos

Un conflicto es **cuando dos transacciones distintas ejecutan instrucciones sobre un mismo ítem X, y al menos una de las dos instrucciones es una escritura**:

- ($R_{Ti}(X)$, $W_{Tj}(X)$): una transacción escribe un ítem que otra leyó
- ($W_{Ti}(X)$, $R_{Tj}(X)$): una transacción lee un ítem que otra escribió
- ($W_{Ti}(X)$, $W_{Tj}(X)$): dos transacciones escriben un mismo ítem

Todo par de instrucciones consecutivas (I1, I2) de un solapamiento que no constituye un conflicto puede ser invertido en su ejecución.

Grafo de precedencias

Es un grafo dirigido donde cada nodo representa una transacción y cada arco Ti→Tj indica que hay un conflicto tal que Ti debe ejecutarse antes que Tj para mantener la equivalencia por conflictos.

- Se crea un nodo por cada transacción
- Se agrega un arco entre los nodos Ti y Tj si y sólo si existe algún conflicto

Cada arco representa una precedencia entre Ti y Tj: en la ejecución serial Ti debe ejecutarse antes que Tj

Un orden de ejecución es serializable por conflictos si y sólo si su grafo de precedencias **no tiene ciclos**

Control de concurrencia

Hay dos enfoques en el control de concurrencia para garantizar el aislamiento:

- Enfoque pesimista: garantizar que no hayan conflictos: poner locks antes de que haya problemas
- Enfoque optimista: consiste en “dejar hacer” a las transacciones y hacer rollback si en la fase de validación se descubre un conflicto.

Control de concurrencia basado en locks

Se trata de usar locks en recursos para no permitir que más de una transacción lo use al mismo tiempo.
El SGBD es quien maneja, inserta los locks.

Un lock tiene dos primitivas de uso:

- Acquire(X) o Lock(X)
- Release(X) o Unlock(X)

Hay dos tipos de locks principales:

- Lock de escritura: acceso exclusivo
- Lock de lectura: acceso compartido

Protocolo de lock de dos fases (2PL)

Una transacción podría adquirir un lock sobre un ítem para leerlo, luego liberarlo, y más tarde volver a adquirirlo para leerlo y modificarlo. Si en el intervalo otra transacción lo lee y escribe, podría producirse la anomalía de la lectura no repetible.
El protocolo para la adquisición y liberación de locks 2PL dice lo siguiente:
Una transacción no puede adquirir un lock luego de haber liberado un lock que había adquirido.

Esta regla divide en dos fases a la ejecución de la transacción:

- Fase de adquisición de locks
- Fase de liberación de locks

Si todas las transacciones cumplen 2PL, la ejecución es serializable.

Surgen dos problemas: Deadlocks e Inanición o postergación indefinida: livelock

Deadlocks

Deadlocks: un conjunto de transacciones quedan cada una de ellas bloqueada a la espera de recursos que otra de ellas posee.

- Prevención de deadlocks:
    1. Cada transacción toma todos los locks al inicio, antes de ejecutar.
    2. Definir un ordenamiento de los recursos, y obligar a que todas las transacciones respeten dicho ordenamiento en la adquisición de locks.
    3. Métodos basados en timestamps
- Detección de deadlocks: mecanismos que identifican un deadlock cuando ya ocurrió, y lo resuelven.
    1. Analizar el *grafo de alocación de recursos* (grafo dirigido con transacciones y los recursos como nodos, en el cual se coloca un arco de una transacción a un recurso cada vez que una transacción espera por un recurso, y un arco de un recurso a una transacción cada vez que la transacción posee el lock de dicho recurso.): si tiene un ciclo, hay deadlock y se aborta alguna transacción del ciclo.
    2. Definir un timeout: si una transacción no logra adquirir un lock en un tiempo límite, se aborta automáticamente.

Inanición

Inanición: ocurre cuando una transacción no logra ejecutarse por un periodo de tiempo indefinido.

- La solución más común consiste en encolar los pedidos de locks, de manera que las transacciones que esperan desde hace más tiempo por un recurso tengan prioridad en la adquisición de su lock.

2PL básico: *Tenemos dos fases, en la primera adquirimos locks, en la segunda los liberamos.*

2PL Conservador: *Bloquea con antelación todos los recursos.*

Estricto S2PL: *Una transacción no puede adquirir un lock luego de haber liberado uno que había adquirido, y los locks de escritura sólo pueden ser liberados después de haber commiteado la transacción.*

Riguroso R2PL: *no diferencia los tipos de locks. Los locks sólo pueden ser liberados después del commit.*

Control de concurrencia basado en timestamps

Se asigna a cada transacción Ti un timestamp TS que debe ser único y determinarán el orden serial respecto al cual el solapamiento deberá ser equivalente.

Se permite la ocurrencia de conflictos, pero siempre que las transacciones de cada conflicto aparezcan de acuerdo al orden serial equivalente:
(WTi(X), RTj(X)) → TS(Ti) < TS(Tj)

Implementación

Para cada ítem X se guarda:

- read_TS(X): es el TS de la transacción más joven que **leyó** el ítem X.
- write_TS(X): es el TS de la transacción más reciente que **escribió** el ítem X.

Cómo funciona:

1. Cuando una transacción Ti quiere **leer** el ítem X. Quiere hacer R(X):
    - Si una transacción más nueva ya escribió X, entonces Ti **se aborta**. Si voy a leer pero si soy mas viejo que el que escribio, aborto.
    - Si no, se actualiza **read_TS(X)** con el timestamp de Ti, y **lee** el valor.
2. Cuando una transacción Ti quiere **escribir** el ítem X. Quiero hacer W(X):
    - Si ya hubo una lectura o escritura posterior (hecha por una transacción más nueva), entonces Ti **se aborta**.
    - Si no, se actualiza **write_TS(X)** con el timestamp de Ti, y **escribe** el valor.

Snapshot Isolation

Cada transacción trabaja con una "foto" (**snapshot**) del estado de la base de datos en el momento en que empieza. O sea que ve una versión fija y consistente de los datos: todas las cosas commiteadas al comienzo de la transacción.

<u>Ventaja:</u> Permite un mayor solapamiento, ya que lecturas que hubieran sido bloqueadas utilizando locks, ahora siempre pueden realizarse.

<u>Desventajas:</u> Mayor consumo de recursos, ya que se deben mantener múltiples versiones de los datos. Cuando hay conflictos de WW, hay que deshacer una de las transacciones del conflicto.

Cuando hay conflicto de WW, generalmente gana aquella que hace primero su commit, mientras que la otra deberá ser abortada

Igualmente no alcanza para garantizar la serializabilidad. Debe combinarse con:

- **Validación con grafos de precedencia**, para evitar ciclos por conflicto de RW
- **Locks de predicados**, que ayudan a detectar conflictos, para detectar precedencias

Niveles de aislamiento

SQL permite definir el nivel de aislamiento de las transacciones con el comando:

```
SET TRANSACTION ISOLATION LEVEL
READ UNCOMMITTED | READ COMMITTED | REPEATABLE READ | SERIALIZABLE;
```

Uno más restrictivo que el que le sigue

Y para definir una transacción:

```
START TRANSACTION [ISOLATION LEVEL ...]
.... (sql commands )
COMMIT | ROLLBACK
```

Según el nivel de aislamiento elegido, pueden producirse o no ciertas anomalías:

1. *Read Uncommitted:* Es la carencia total de aislamiento: No se emplean locks, y se accede a los ítems sin tomar ninguna precaución.
2. *Read Committed:* Evita la anomalía de lectura sucia.
3. *Repeatable Read:* Evita la lectura no repetible y la lectura sucia.
4. *Serializable:* Evita todas las anomalías, y asegura que el resultado de la ejecución de las transacciones es equivalente al de algún orden serial.

Nivel de aislamiento	Lectura sucia	Lectura no repetible	Fantasma
READ UNCOMMITTED	✗	✗	✗
READ COMMITTED	✓	✗	✗
REPEATABLE READ	✓	✓	✗
SERIALIZABLE	✓	✓	✓